

C-MEDIUM-1：位级操作

（四个MEDIUM里面这个最对胃口（美味啊！）所以这一题整个活。）

——现在是星历星历3024年7月21日4时20分，我是微光计划313号工程师，已确认飞船情况达到伽马级危机标准，申请启动三号条约，直接意识接入格里姆神经中枢控制系统。

——收到，三号条约已启用，允许使用二级密钥进入GLIMMER核心区域。

神经一阵刺痛，像是有能量波顺着神经末梢直接灌向大脑，挤在我的头骨里，不过很快它们就从我脑袋里退下了，仿佛超越了物理的阻隔，带着我的思维四散开来冲向一片很宽很大的地方。

主舱室，GLIMMER引导区，我是微光计划313号工程师。

.....

模块一：启动引导区 —— 语言的本源

——正在通过数字密钥验证.....

——警告！警告！密钥接口状态异常！无法定位接口信标！

——执行人工核验程序。

——单向对接路径已生成，反密系统正常运行中.....

.....

我拿起舰长手册，虽然刚刚醒来记忆还很模糊，很多东西暂时想不起来，不过手指已经很懂事地翻到了密码位置，舱外巨大恒星的红光与舱内警报灯把两个数字照得很亮。211，10110110。

$211-128=83-64=19-16=3-2=1$ $2+4+16+32+128=182$

211=11010011

10110110=182

顺利地通过了引导区（虽然在强辐射影响下并没有起到什么引导的作用），沿着应急路径自动深入，我的思维渐渐放空，想到了一些奇奇偶偶的事情。

如果我要判断奇偶直接%2看是0是1不就好了嘛。

不对，不够快。记忆慢慢回流，我想起了数字的构成方式，想起曾经使用过一种神奇的运算，既然是二进制，就直接每一位做运算吧。

$\text{num}\&1==1$ 即为奇数，反之是偶数。

.....

模块二：能量阵列 —— 逻辑门的重构

我又想起了一些事情。

地球能源几近枯竭，为了解决能源危机，人类派出了数支星际舰队，去寻找传说中藏匿于茫茫星海的无尽能源，代号：微光计划。

我当时在跟几个同事玩上个招新题的那个五子棋，很显然没有人喜欢告别家乡跑到太空里受罪。

我们决定谁输了谁去。

.....

——已进入能量模块。

飞船上能量分布极不均匀，主发电机连接系统故障，几乎所有的能量都集中在冷冻舱，发动机能量供给小于最低限度，飞船直愣愣地冲向巨大的恒星，像人类肘击水泥地一样。

六个位运算符是六种精密的武器，我准备好数以万计地操控能量了。首先看报错提示：

rest1: 比较a和b的每一项只有第四位都是1，所以结果是1000=8;

rest2: 第1、3、4、5位有1，所以结果是11101=29;

rest3: 第1、3、5位不同，所以结果是10101=21;

rest4: a变成了00110000，b变成了00001100，第3、4、5、6位有1，所以结果是111100=60。

下面对其进行修复。

```
#include <stdio.h>

void prepare(int* x,int* n){
    scanf("%d",x);
    scanf("%d",n);
}

int task2(int x,int n){
    x=x>>(n-1);
    return(x&1);
}

int task3(int x,int n,int t){
    int t1=1<<(n-1);
    if(t==1){return x|t1;}
    else{return x&(~t1);}
}

int task4(int x){
    return (x&(-x));
}

int task5(unsigned int a,unsigned int b){
    int i=0;
    while(a!=b && i<100){
        a>>=1;
        b>>=1;
        i++;
    }
    return a<<i;
}

int main(){
    int x,n,t,a,b;

    prepare(&x,&n);
    printf("%d",task2(x,n));

    prepare(&x,&n);
```

```

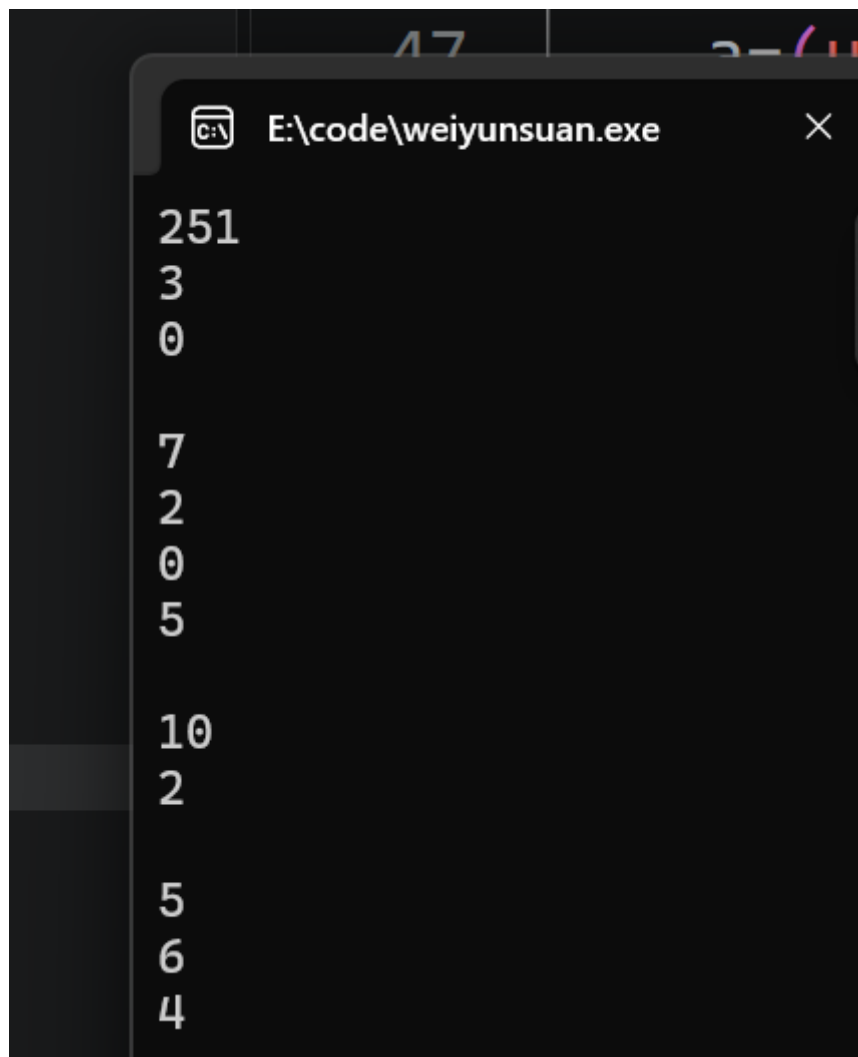
scanf("%d",&t);
printf("%d",task3(x,n,t));

scanf("%d",&x);
printf("%d",task4(x));

prepare(&a,&b);
a=(unsigned int)a;
b=(unsigned int)b;
printf("%d",task5(a,b));

while(getchar() != '\n');
getchar();
return 0;
}

```



task2: 运用上个题末尾的思考，此题等价于将数字右移至指定位后判断奇偶。

——输能模块故障已定位，正在执行自动修复程序。

task3: 1001的不太好办，分个类吧，先把数字1移到指定位置，1就直接|改掉，0得先取反再&。

——密钥已接入，系统运行正常。

task4: 我突然想起学指针的时候有一个涉及位运算的题，急头白脸想了半个小时（不嘻嘻），总之就是非常经典。

——锚点已设定于指定位置，即将执行数据重构。

task5：只要在ab中间发生了进位就必然出现0，只有高位的1可以保存下来，因此只需找到ab的共同“前缀”，通过不断左移比较，再加计数器进行最后的右移即可。

——已排除冗余数据流，重构时可最大限度保护源码。

——正在执行人工位运算引导，借由已有的逻辑模块，对位运算法则进行重构……

——1，逻辑运算与位运算皆使用与或非门物理硬件，底层本质相同。而逻辑运算直接将操作数视为true和false，位运算则是逐位比较。此外逻辑运算有短路特性，&&左边为假就不再计算。

——2，按位与、或、异或均有结合律和交换律，按位与和或之间有分配律（按其定义很好理解）。按位异或中一个数的某一位异或1结果改变，异或0结果不变，异或自己变0。左移右移可看作乘除2，执行n次（有小数直接抹了）。

——3，补码=反码+1，其高位溢出机制可保留正负数的可加性，因此算术右移会高位补会符号位，保证结果是除以二，逻辑右移是严格的右移。

——已确认运算法则细节，能量模块修复中……

——剩余能量统计完毕，输能模块已恢复正常。

令人安心的蓝光蒙盖在控制台上，信号曲线在恒星喷发的粒子流里扭动，像是在钢丝上舞蹈。

尚存一线生机。

.....

模块三：通信模块 —— 无碰撞协议

——预计逃逸轨迹超出飞船算力范围，请求向母星基站申请数据援助。

——执行。

为确保压缩信息流中数据不至湮灭，我需要分隔几个危险指令。

```
int hasCommonChar(const char *s1, const char *s2) {
    int mask1 = 0;
    int mask2 = 0;
    for(int i=0;s1[i]!='\0';i++){
        mask1|(1<<(s1[i]-'a'));
    }
    for(int i=0;s2[i]!='\0';i++){
        mask2|(1<<(s2[i]-'a'));
    }
    if(mask1&mask2){return 1;}
    return 0;
}
```

我一开始没有想清楚一个字节怎么存入茫茫多的数据，要是字符串很长的话不就放不下了吗？犹豫间飞船被喷发出的高温气体击中，我突然想到只需要存入每个出现的字母，如果已有就不变……好像可以|1。用mask1和mask2记录下来然后&一下！

——信息已同步，母星将会帮助我们计算逃逸轨道。

——警告！刚刚的高温气体喷发干扰了反应堆的正常进行，请立即执行重启程序！

还有紧急停堆？！

模块四：反应堆核心 —— 绝对平衡的艺术

好可怕……控制棒散落一地，还好紧急停堆了。要是不小心插错一根，一定、一定会炸的吧！

试着摆了一下三根，四根，五根的情况，发现数量上来了过后摆完一根要给下面的格子画上叉不然不知道哪里可以放了。

这个思路可以自然的移植到计算机上，设计一个while循环从这一排的第一个开始试。

限制条件分成三种，同列，“/”形斜线和“\”形斜线，借助|和移动完成。

本来还想用a[n]和两个for解决的，但是要么卡死在一层要么蹬蹬蹬地往下面走完全回溯不了，只能使用递归了。

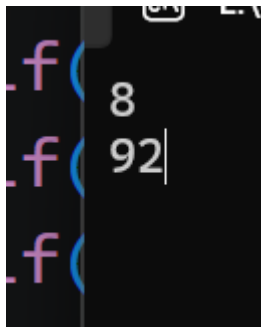
传入往下走的指针，三个针对下一层的限制条件和一个计数器，返回后把指针减回来。

```
#include <stdio.h>
#include <stdlib.h>
int n;
int num=0;

void QUEEN(int* a,int k,int g,int b,int j){
    for(int i=0;i<n;i++){
        *a=1<<i;
        if(*a&k){continue;}
        if(*a&g){continue;}
        if(*a&b){continue;}
        if(j==n-1){
            num++;
            return;}
        else{
            QUEEN(++a,k|*a,(g|*a)<<1,(b|*a)>>1,j+1);
            a--;
        }
    }
}

int QueenNum(int n){
    int* a=malloc(sizeof(int)*n);
    QUEEN(a,0,0,0,0);
    return num;
}

int main(){
    scanf("%d",&n);
    printf("%d",QueenNum(n));
    while(getchar() != '\n');
    getchar();
    return 0;
}
```



——15根控制棒已按2279184种方案重新排序，反应堆运行正常，准备启动能量超载。

——母星通讯正常联通，检测到传入数据量过大，已存入GLIMMER管道系统中。

——投放M.O.U.S.E获取数据。

——收到。

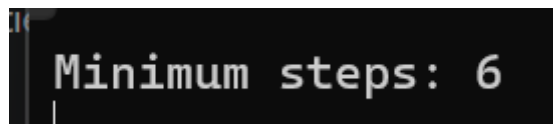
模块五：管道穿梭 —— 自动化寻路

我的记忆渐渐恢复了。

我想起我上高中的时候，老师介绍信息竞赛，演示里也是一个寻路的题，红色方块从起点开始向波纹一样扩散。

水利万物而不息。

```
while(front<rear){
    if(queue[front].pos==target){
        return queue[front].dist;
    }
    int c_dis=queue[front].dist;
    int c_pos=queue[front++].pos;
    for(int i=0;i<4;i++){
        int a=c_pos/4+dr[i];
        int b=c_pos%4+dc[i];
        if(a>=4||a<0||b>=4||b<0){continue;}
        int pos=a*4+b;
        if(walls&(1<<pos)||visited&(1<<pos)){continue;}
        visited|=(1<<pos);
        queue[rear++]=Node{pos,c_dis+1};
    }
}
```



扩散，碰撞，弹回，入列，queue中刚好是同一个dis的数据排完了才用到下一个……

——路径已同步，确认引力弹弓效应符合预期。

……我惊叹于它的神奇。

赤红的恒星在身后渐渐黯淡。

刚才我还在害怕它那炽热的喷流打碎驾驶舱的玻璃，现在它已经变成了一个橘黄的光点，一颗随处可见的星星。

难得被唤醒，我想多在驾驶舱的柔软座位上多躺一会，随便数会星星。

那一小段时间非常清闲。

我想，人类文明是从那些和我一样清闲的人，找了一个和我一样暖和的地方躺着数星星开始的。

在我面前的驾驶舱里，GLIMMER也在静悄悄地用她的方法数数，把1和0变来变去，像是枕边的恋人在轻盈的呼吸。

我又想起来，其实我当初并没有输掉那几盘五子棋。